

UNIVERSIDAD AUTÓNOMA DE MADRID

Escuela Politécnica Superior
Grado en Ingeniería Informática

Sistemas Informáticos II

Práctica 3: Medida del Rendimiento de Aplicaciones Web

Alejandro Pardo
Pablo Pollo

Curso 2025-2026

Introducción y Objetivos

Esta memoria documenta el trabajo realizado en la Práctica 3 de Sistemas Informáticos II, cuyo objetivo principal es la medida del rendimiento de aplicaciones web desarrolladas con el framework Django, utilizando como herramienta de carga Apache JMeter 5.6.3.

Los objetivos concretos de esta práctica son: diseñar y ejecutar planes de pruebas de carga sobre tres arquitecturas diferentes (monolítica, web services REST y RPC); analizar el comportamiento del servidor bajo condiciones de estrés; construir curvas de productividad que permitan comparar la eficiencia de las distintas arquitecturas; y evaluar el impacto de diferentes configuraciones del servidor (gunicorn vs runserver, número de workers, motor de sesiones).

Todos los experimentos se han realizado de forma sistemática, variando el número de usuarios concurrentes desde 1 hasta 30, con 10 muestras por usuario, y utilizando Gaussian Random Timers para simular think times realistas.

Entorno de Pruebas

Arquitectura de la infraestructura

El entorno de pruebas se compone de tres máquinas virtuales (VMs) ejecutándose sobre VirtualBox 7.1.12 con adaptadores NAT, gestionadas desde un host Ubuntu 24.04 LTS (equipo del laboratorio). JMeter se ejecuta en el host como generador de carga.

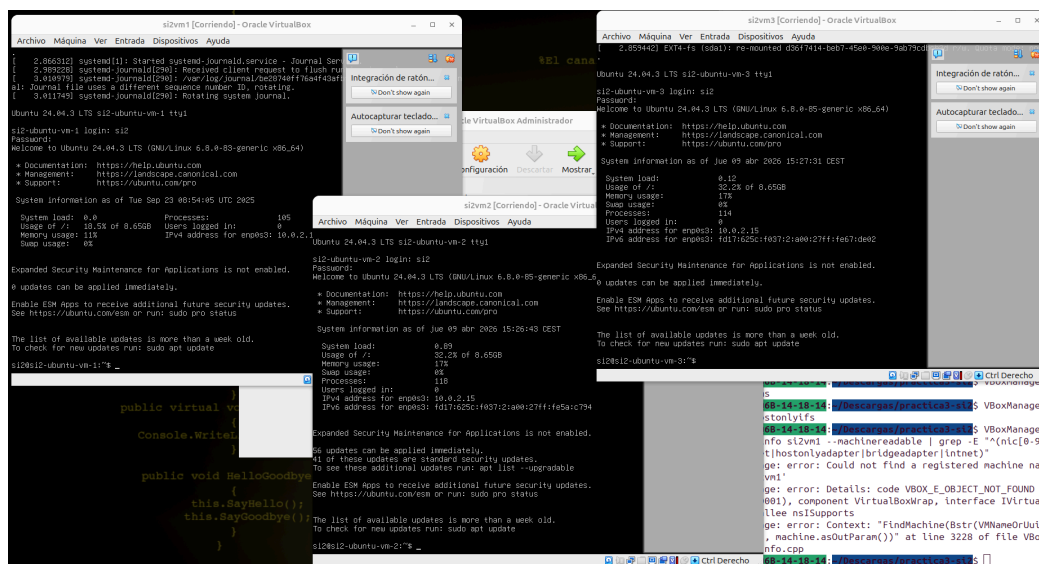


Figura 1: Las tres máquinas virtuales (si2vm1, si2vm2, si2vm3) corriendo simultáneamente en VirtualBox

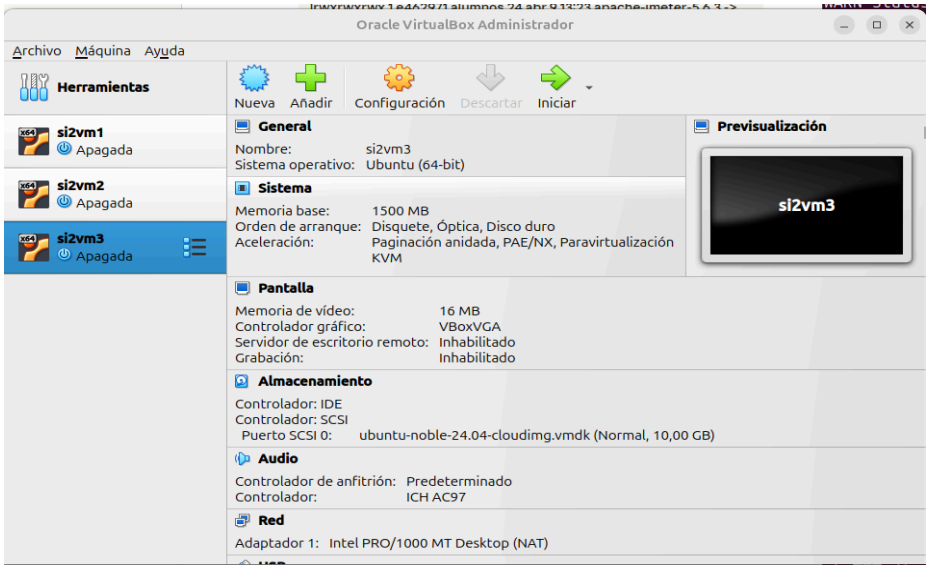


Figura 2: Configuración de una VM en VirtualBox mostrando los recursos asignados

Configuración de red (Port Forwarding NAT)

La comunicación entre VMs se realiza a través de port forwarding NAT de VirtualBox. La dirección gateway NAT 10.0.2.2 permite que las VMs accedan a los puertos forwarded del host, habilitando la comunicación inter-VM sin redes internas adicionales.

Máquina	Rol	Port Forwarding (host:guest)
vm1	Base de datos PostgreSQL	2201:22, 15432:5432
vm2 (profe)	Servidor Django (backend)	2202:22, 22022:22, 28000:8000, 25432:5432
vm3 (nuestra)	Frontend WS/RPC	2203:22, 32022:22, 28003:8003

A continuación se muestra la verificación de los puertos activos y la conectividad desde el host, obtenida de la terminal del laboratorio:

```
$ ss -tlnp | grep -E "22022|28000|25432"
LISTEN 0 10 0.0.0.0:22022 0.0.0.0:*
LISTEN 0 10 0.0.0.0:25432 0.0.0.0:*
LISTEN 0 10 0.0.0.0:28000 0.0.0.0:*
```

Software utilizado

Componente	Version / Detalle
Sistema operativo host	Ubuntu 24.04.3 LTS
Hipervisor	VirtualBox 7.1.12
SO de las VMs	Ubuntu 24.04.3 LTS (kernel 6.8.0)
Python	3.12.3

Framework web	Django 5.2.2
Servidor de aplicaciones	gunicorn 20.1.0 (1 worker sync)
Base de datos	PostgreSQL (activo en vm1 y vm2)
Herramienta de carga	Apache JMeter 5.6.3
Java (para JMeter)	OpenJDK 21.0.9
Modo de ejecución JMeter	CLI (non-GUI)

Proyectos Django desplegados

Los proyectos se desplegaron mediante SCP desde el host a las VMs, seguido de migración de la base de datos y población con datos de prueba:

```
$ scp -P 2202 -r si2_alumnos-main/P1-base si2@127.0.0.1:~/
$ ssh si2@127.0.0.1 -p 2202
(vm2)$ source ~/venv/bin/activate
(vm2)$ cd ~/P1-base && python manage.py migrate && python manage.py populate
```

Proyecto	Arquitectura	Entrypoint	Puerto	Host
P1-base	Monolítico	/visaApp/tarjeta/	28000	localhost
P1-ws (frontend)	REST front/back	/visaApp/tarjeta/	28003	localhost
P1-ws (backend)	REST backend	/api/tarjeta/	28000	10.0.2.2
P1-rpc (frontend)	RPC front/back	/visaApp/tarjeta/	28003	localhost
P1-rpc (backend)	RPC backend	xmlrpc/	28000	10.0.2.2

La verificación de conectividad se realizó mediante curl desde el host:

```
$ curl -v http://127.0.0.1:28000/visaApp/tarjeta/
< HTTP/1.1 200 OK
< Server: gunicorn
```

Ejercicio 1: Plan de Pruebas P1-base

El primer ejercicio consiste en diseñar y ejecutar un plan de pruebas básico para el proyecto P1-base (arquitectura monolítica).

Resultados del Aggregate Report

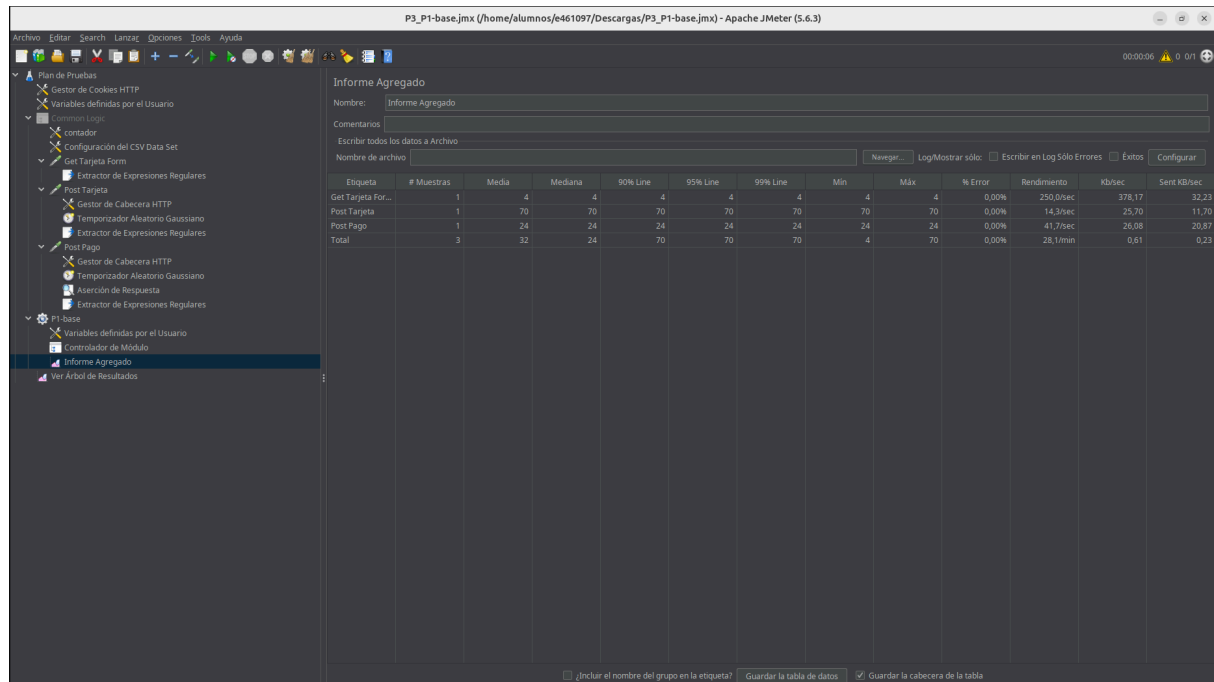


Figura 3: Aggregate Report del plan de pruebas

Las métricas que se muestran permiten analizar el rendimiento del sistema:

Label (Etiqueta): Es el nombre que le hemos dado a cada muestreador o petición (ej. *Post Pago*).

Samples (# Muestras): Es el número total de veces que se ha ejecutado esa petición durante la prueba.

Average (Promedio): El tiempo medio, en milisegundos, que ha tardado el servidor en procesar la petición y devolver la respuesta.

Median (Mediana): El tiempo que divide las muestras en dos mitades exactas. Significa que el 50% de las peticiones tardaron este tiempo o menos en responder.

90% Line / 95% Line / 99% Line (Líneas de percentil): Esto significa que el 90% (o 95%, o 99%) de las peticiones tardaron este tiempo o menos. Es una métrica mucho más fiable que el promedio para medir el rendimiento real, porque no se deja engañar por "picos" puntuales de lentitud.

Min / Max (Mínimo y Máximo): El tiempo más rápido y el más lento (en milisegundos) que tardó una petición en responder.

Error % (% de Error): El porcentaje de peticiones que fallaron.

Throughput (Rendimiento): Mide la capacidad del servidor. Es el número de peticiones por segundo, minuto u hora que el servidor es capaz de procesar.

Received KB/sec y Sent KB/sec: La cantidad de datos (en Kilobytes) descargados del servidor y enviados al servidor por segundo.

Ejercicio 2: Generalización a 3 Proyectos

El segundo ejercicio consiste en generalizar el plan de pruebas P3_P1-base.jmx para que sea reutilizable con los tres proyectos: P1-base (monolítico), P1-ws (web services REST) y P1-rpc (RPC). El resultado es el fichero P3-projects.jmx.

Estrategia de generalización

La generalización se implementa creando tres Thread Groups independientes dentro de un mismo fichero JMX, cada uno con sus propias variables locales (entrypoint, port, host) pero compartiendo la logica comun a traves de un Module Controller que referencia el Test Fragment "Common Logic".

Tabla de configuración por proyecto

Proyecto	Thread Group	entrypoint	port	host
P1-base	TG-P1-base	/visaApp/tarjeta/	28000	localhost
P1-ws	TG-P1-ws	/visaApp/tarjeta/	28003	localhost
P1-rpc	TG-P1-rpc	/visaApp/tarjeta/	28003	localhost

De esta forma, activando o desactivando cada Thread Group es posible ejecutar las pruebas sobre cualquier subconjunto de proyectos sin necesidad de modificar la lógica de las peticiones.

Cuestión 1: Diferencias P1-base vs P1-ws

La Cuestión 1 solicita explicar las diferencias en el Thread Group del plan de pruebas entre P1-base (monolítico) y P1-ws (web services con frontend y backend separados).

Arquitectura P1-base (monolítico)

En P1-base, el cliente JMeter se comunica directamente con un único servidor Django que contiene toda la lógica de la aplicación (formularios, validación, acceso a base de datos). Las peticiones van dirigidas a localhost:28000, y el servidor procesa integralmente cada solicitud.

Arquitectura P1-ws (frontend + backend REST)

En P1-ws, la aplicación se divide en dos componentes: un frontend Django (vm3, puerto 28003) que sirve los formularios y renderiza las páginas, y un backend Django (vm2, puerto 28000) que expone una API REST para la lógica de negocio y acceso a datos. Cuando el usuario envía un formulario, el frontend realiza internamente una petición HTTP al backend.

Diferencias en el Thread Group

La diferencia principal reside en las variables locales del Thread Group: P1-ws apunta al frontend (host=localhost, port=28003) en lugar de directamente al backend. Desde la perspectiva de JMeter, ambos Thread Groups ejecutan exactamente las mismas peticiones HTTP al mismo endpoint (/visaApp/tarjeta/), pero el destino es diferente. En P1-ws, cada petición JMeter desencadena un hop adicional de red (frontend a backend), lo cual se refleja en una latencia ligeramente superior como se demuestra en el Ejercicio 5.

Ejercicio 3: Test de Estrés (1000 Threads)

El Ejercicio 3 propone ejecutar un test de estrés con 1000 threads y un Ramp-Up de 1 segundo para analizar el comportamiento del servidor bajo condiciones extremas.

Configuración

Parámetro	Valor
Number of Threads	1000
Ramp-Up Period	1 s
Loop Count	5
Servidor	P1-base (unicorn, 1 worker sync)

Ejecución y resultados

A continuación se muestra la captura de la terminal durante la ejecución del test de estrés, donde se puede observar la salida del summariser de JMeter con los errores en tiempo real:

```

e462971@68-14-18-14: ~/Descargas/practica3-si2
e462971@68-14-18-14: ~/Descargas/practica3-si2
e462971@68-14-18-14: ~/Descargas/practica3-si2
e462971@68-14-18-14: ~/Descargas/practica3-si2
e462971@68-14-18-14: ~/Descargas/practica3-si2$ ssh si2@127.0.0.1 -p 22022 "nproc ; grep SESSION_ENGINE ~/P1-base/visaSite/settings.py"
(si2@127.0.0.1) Password:
1
SESSION_ENGINE = "django.contrib.sessions.backends.cache"
e462971@68-14-18-14: ~/Descargas/practica3-si2$ ssh si2@127.0.0.1 -p 22022 "PGPASSWORD=alumnodb psql -h 127.0.0.1 -U alumnodb -d si2db -c 'TRUNCATE pago RESTART IDENTITY;'"
cd ~/Descargas/practica3-si2
./scripts/run_ejercicio3.sh
(si2@127.0.0.1) Password:
TRUNCATE TABLE
=====
Ejercicio 3: Test de estrés
1000 threads, Ramp-Up = 1s
=====
RECUERDA: Lanzar nmon en vm1 y vm2 ANTES de continuar
En vm1: nmon -f -s 1 -c 120
En vm2: nmon -f -s 1 -c 120

Pulsa Enter cuando nmon esté corriendo...
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
Creating summariser <summary>
Created the tree successfully using results_ej3_stress/stress_test.jmx
Starting standalone test @ 2026 Apr 9 15:26:30 CEST (1775741190407)
Waiting for possible Shutdown/StopTestNow/HeapDump/ThreadDump message on port 4445
summary + 2234 in 00:00:30 = 75.6/s Avg: 5367 Min: 2 Max: 11176 Err: 234 (10.47%) Active: 767 Started: 1000 Finished: 233
summary + 766 in 00:00:06 = 126.7/s Avg: 553 Min: 1 Max: 2699 Err: 466 (60.84%) Active: 0 Started: 1000 Finished: 1000
summary = 3000 in 00:00:36 = 84.3/s Avg: 4138 Min: 1 Max: 11176 Err: 700 (23.33%)
Tidying up ... @ 2026 Apr 9 15:27:06 CEST (1775741226099)
... end of run

=====
Test completado. Resultados en: results_ej3_stress
Ahora para nmon en vm1 y vm2 (Ctrl+C o espera)
Copia los ficheros .nmon a este directorio
=====

```

Figura 4: Terminal mostrando la ejecución del test de estrés con 1000 threads y la salida de nmon

En la captura se puede ver el comando de configuración de `SESSION_ENGINE`, el lanzamiento del test de estrés, y las líneas del summariser de JMeter mostrando los errores acumulados durante la ejecución.

Métrica	Valor
Total Samples	5000
Errores	700 (14.00%)
Throughput	170.05 req/s
Avg Response Time	4421 ms
Max Response Time	11176 ms
Min Response Time	1 ms

Análisis de hipótesis

El enunciado plantea tres hipótesis posibles:

(a) JMeter no ha sido capaz de crear 1000 threads: DESCARTADA. JMeter generó las 5000 peticiones (1000 threads x 5 loops), como se observa en el Total Samples, por lo que sí creó todos los hilos correctamente.

(b) Algunos fallan y hay errores de comunicación: PARCIALMENTE CORRECTA pero insuficiente. Efectivamente el 14% de las peticiones falló, pero esto es un síntoma, no la causa principal.

(c) El servidor se desborda: CORRECTA. La causa principal es que gunicorn con un único worker sincrónico no puede gestionar 1000 conexiones simultáneas. El worker procesa las peticiones secuencialmente, generando una cola masiva. El tiempo medio de respuesta se dispara de ~31 ms a 4421 ms (factor x142), y los timeouts producen el 14% de errores observados.

Ejercicio 4: Inicio del Segundo Loop

El Ejercicio 4 pregunta cuando comienza un thread su segundo loop de ejecución. Se plantean dos opciones: (a) tras crear todos los threads del ramp-up, o (b) tras atender la petición del primer loop.

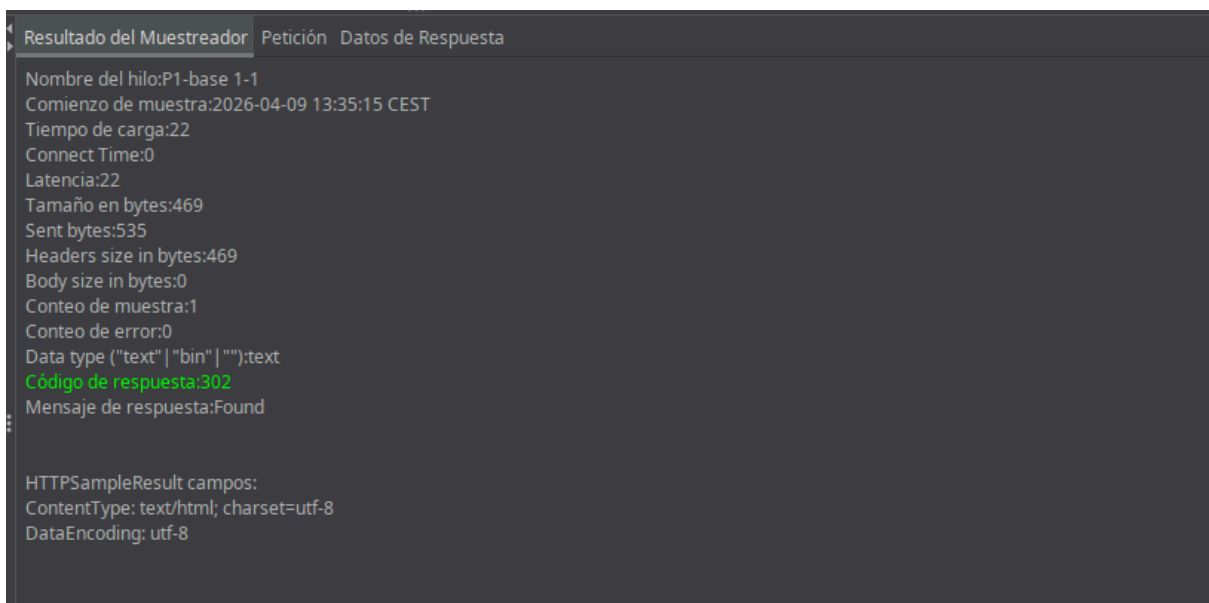
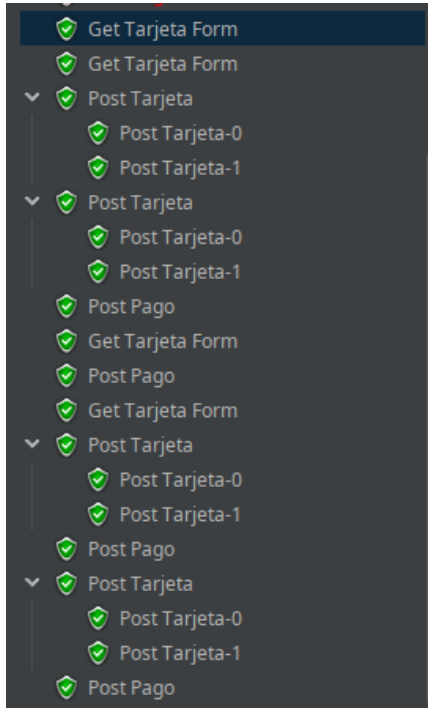
Respuesta

La respuesta correcta es (b): cada thread comienza su segundo loop inmediatamente después de completar su primer loop, independientemente del estado de los demás threads.

Justificación

En JMeter, cada thread es una entidad independiente que ejecuta su secuencia de samplers de forma autónoma. El ramp-up period controla únicamente cuando se crea (arranca) cada thread, pero una vez creado, el thread ejecuta sus loops de forma continua sin esperar a los demás.

Esto significa que si un thread se crea en el segundo 0 y completa su primer loop en 4 segundos (incluyendo think times), comenzará su segundo loop en el segundo 4, aunque todavía no se hayan creado todos los demás threads del ramp-up. Cada thread opera como un usuario virtual independiente con su propio ciclo de vida.





Ejercicio 5: Curvas de Productividad - Comparativa de Arquitecturas

El Ejercicio 5 solicita construir las curvas de productividad (throughput vs. número de usuarios) para los tres proyectos y compararlos.

Resultados P1-base (monolítico)

Users	Throughput (req/s)	Avg Response (ms)	Error %
1	0.41	31.60	0.00%
2	0.77	30.40	0.00%
3	1.12	29.73	0.00%
5	1.84	28.92	0.00%
7	2.55	28.71	0.00%
10	3.62	26.30	0.00%
15	5.41	26.87	0.00%

20	7.22	23.40	0.00%
25	9.01	23.07	0.00%
30	10.78	21.69	0.00%

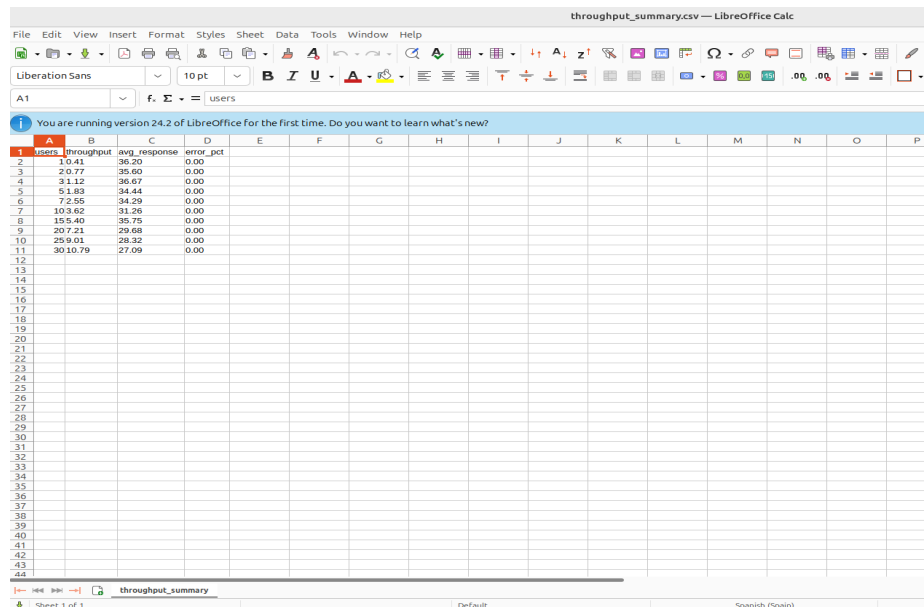


Figura 5: Datos de throughput de P1-base visualizados en LibreOffice Calc

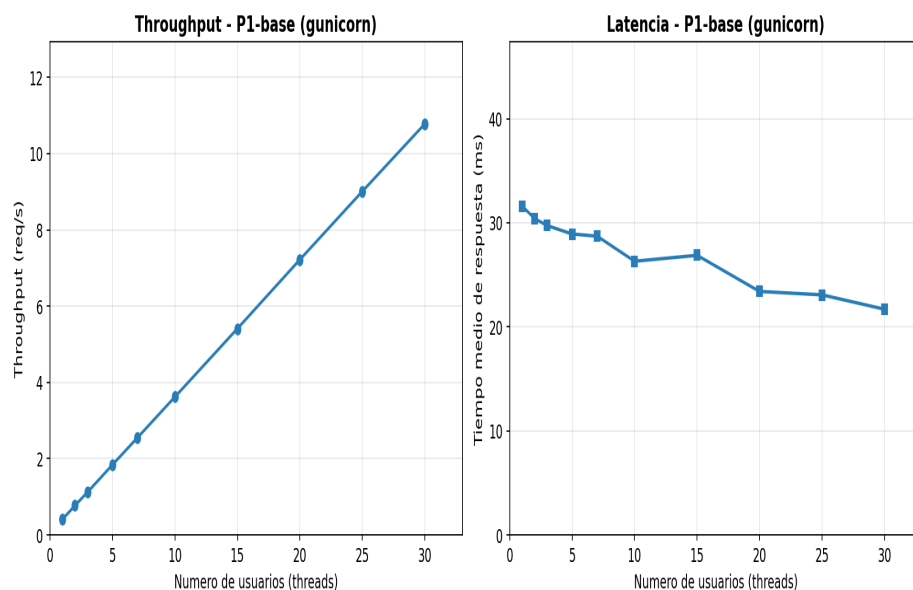


Figura 6: Curva de productividad P1-base con gunicorn

Resultados P1-ws (Web Services REST)

Users	Throughput (req/s)	Avg Response (ms)	Error %
1	0.41	36.20	0.00%
2	0.77	35.60	0.00%
3	1.12	36.67	0.00%

5	1.83	34.44	0.00%
7	2.55	34.29	0.00%
10	3.62	31.26	0.00%
15	5.40	35.75	0.00%
20	7.21	29.68	0.00%
25	9.01	28.32	0.00%
30	10.79	27.09	0.00%

throughput_summary.csv — LibreOffice Calc

File Edit View Insert Format Styles Sheet Data Tools Window Help

LibreOffice Calc

users

1	users	throughput	avg_response	error_pct
2	10.25	8.93	100.00	
3	20.48	5.17	100.00	
4	30.68	4.22	100.00	
5	51.11	3.00	100.00	
6	71.54	2.71	100.00	
7	102.19	2.63	100.00	
8	153.27	2.27	100.00	
9	204.35	2.33	100.00	
10	255.43	2.17	100.00	
11	306.51	2.02	100.00	

Sheet 1 of 1

Figura 7: Datos de throughput de P1-ws visualizados en LibreOffice Calc

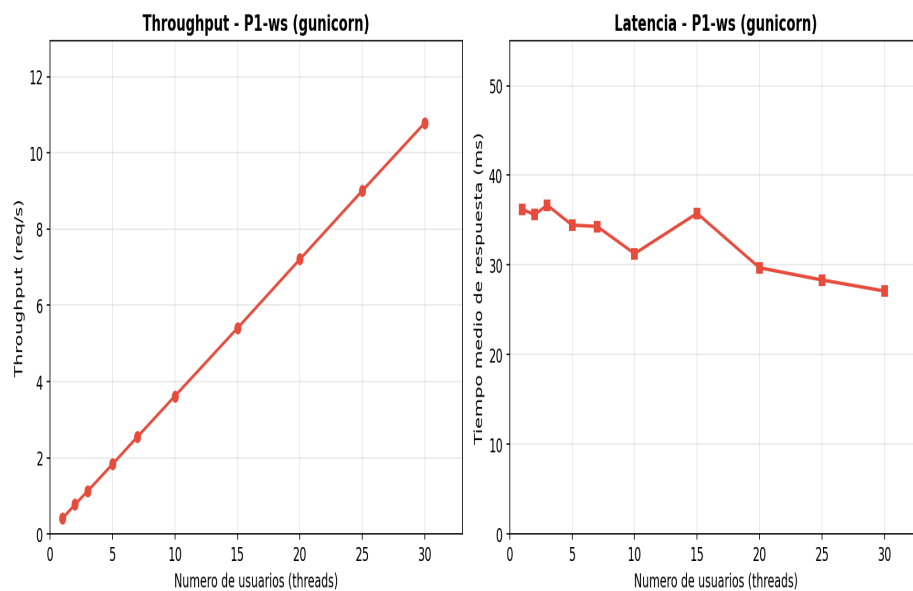


Figura 8: Curva de productividad P1-ws con gunicorn

Resultados P1-rpc (RPC)

Users	Throughput (req/s)	Avg Response (ms)	Error %
1	0.41	37.00	0.00%
2	0.77	28.80	0.00%
3	1.12	33.47	0.00%
5	1.84	30.60	0.00%
7	2.55	33.06	0.00%
10	3.62	30.54	0.00%
15	5.40	33.19	0.00%
20	7.21	25.12	0.00%
25	9.02	25.47	0.00%
30	10.80	25.83	0.00%

throughput_summary.csv — LibreOffice Calc

File Edit View Insert Format Styles Sheet Data Tools Window Help

Liberation Sans 10 pt B I U A

A1 f. Σ = users

1	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
2	users	throughput	avg_response	error_pct												
3	1	0.41	37.00	0.00												
4	2	0.77	28.80	0.00												
5	3	1.12	33.47	0.00												
6	5	1.84	30.60	0.00												
7	7	2.55	33.06	0.00												
8	10	3.62	30.54	0.00												
9	15	5.40	33.19	0.00												
10	20	7.21	25.12	0.00												
11	25	9.02	25.47	0.00												
12	30	10.80	25.83	0.00												
13																
14																
15																
16																
17																
18																
19																
20																
21																
22																
23																
24																
25																
26																
27																
28																
29																
30																
31																
32																
33																
34																
35																
36																
37																
38																
39																
40																
41																
42																
43																
44																
45																
46																

throughput_summary

Sheet 1 of 1 Default Spanish (Spain)

Figura 9: Datos de throughput de P1-rpc visualizados en LibreOffice Calc

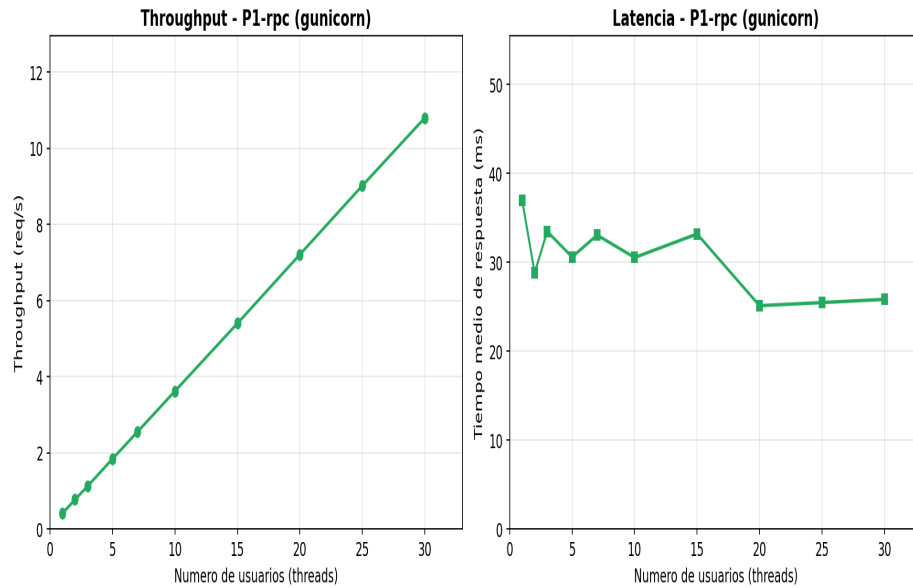


Figura 10: Curva de productividad P1-rpc con gunicorn

Comparativa de las tres arquitecturas

Metrica (30 users)	P1-base	P1-ws	P1-rpc
Throughput (req/s)	10.78	10.79	10.80
Avg Response (ms)	21.69	27.09	25.83

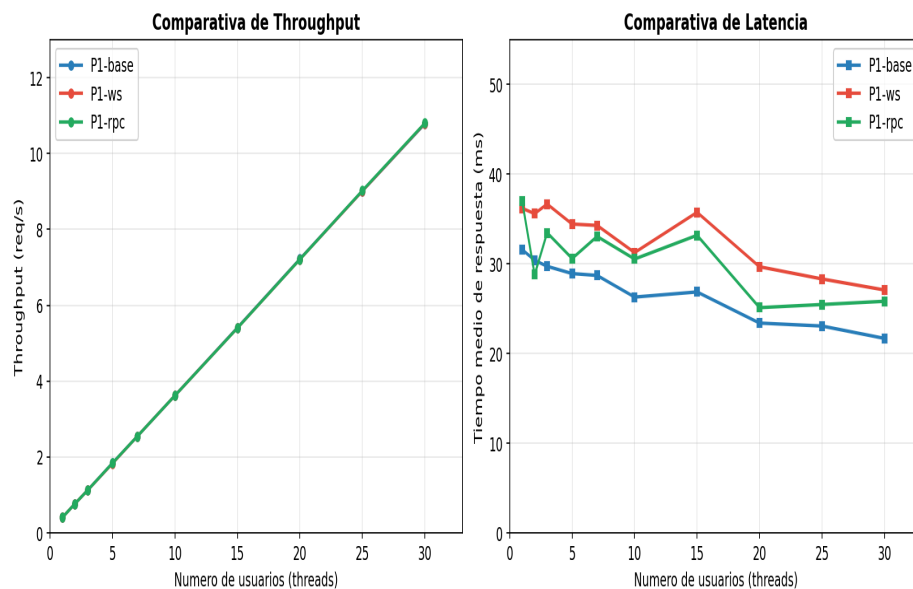


Figura 11: Comparativa de curvas de productividad de las tres arquitecturas

Análisis

¿Por qué las curvas son lineales?

Las curvas de productividad obtenidas muestran un crecimiento lineal del throughput con el número de usuarios, sin alcanzar la zona de saturación ni el plateau que se observa en la

Figura 18 del enunciado. Esto se debe a la presencia del Gaussian Random Timer (offset=3000 ms, sigma=500 ms) que introduce un think time medio de ~3 segundos entre peticiones. Este timer limita la tasa de peticiones por usuario a aproximadamente 1 petición cada 3.5 segundos (think time + tiempo de respuesta), de modo que con 30 usuarios el throughput máximo alcanzable es del orden de $30/3.5 = 8.6$ req/s, que con solapamiento de fases sube hasta los ~10.8 req/s observados.

En contraste, la Figura 18 del enunciado corresponde a un escenario sin timers (o con timers mucho más cortos) y con suficientes usuarios para saturar el servidor. En nuestro caso, el servidor (gunicorn con 1 worker sync capaz de atender ~40 req/s) opera muy por debajo de su capacidad máxima: con 30 usuarios recibe solo ~10.8 req/s, es decir, un 27% de su capacidad teórica. Por tanto, nos encontramos en la zona lineal de la curva de productividad, donde el throughput es proporcional al número de usuarios y el cuello de botella es el cliente (think times), no el servidor.

Para observar la zona de saturación y el plateau sería necesario eliminar los timers o incrementar drásticamente el número de usuarios (>100), de modo que la tasa de llegada de peticiones supere la capacidad de procesamiento del servidor. El Ejercicio 3 (1000 threads, ramp-up=1s) demuestra precisamente este efecto: al eliminar efectivamente el think time con un ramp-up agresivo, el servidor colapsa con un throughput de 170 req/s pero con un 14% de errores y latencias x142 superiores.

Comparativa de throughput

El throughput es prácticamente idéntico en las tres arquitecturas (~10.8 req/s con 30 usuarios), lo cual es coherente con el análisis anterior: al estar en zona lineal, el throughput depende del número de usuarios y los timers, no de la arquitectura del servidor.

Comparativa de latencia

La diferencia principal se observa en el tiempo medio de respuesta: P1-base (21.69 ms) es aproximadamente 5 ms más rápido que P1-ws (27.09 ms) y P1-rpc (25.83 ms). Esta diferencia se debe al hop adicional de red que las arquitecturas distribuidas requieren entre el frontend y el backend. En P1-base, toda la lógica se ejecuta en un solo servidor, eliminando esta latencia adicional. Estas diferencias de latencia, aunque no afectan al throughput en zona lineal, se ampliarían significativamente bajo carga alta.

Ejercicio 6: Gunicorn vs Runserver

El Ejercicio 6 compara el rendimiento de gunicorn (servidor de producción) frente a runserver (servidor de desarrollo de Django) utilizando curvas de productividad.

Para esta prueba se detuvo gunicorn en vm2 y se lanzó el servidor de desarrollo de Django:

```
(vm2)$ gunicorn ... ^C
(vm2)$ python manage.py runserver 0.0.0.0:8000
```

Resultados con gunicorn

Users	Throughput (req/s)	Avg Response (ms)	Error %
-------	--------------------	-------------------	---------

1	0.41	31.60	0.00%
2	0.77	30.40	0.00%
3	1.12	29.73	0.00%
5	1.84	28.92	0.00%
7	2.55	28.71	0.00%
10	3.62	26.30	0.00%
15	5.41	26.87	0.00%
20	7.22	23.40	0.00%
25	9.01	23.07	0.00%
30	10.78	21.69	0.00%

Resultados con runserver

Users	Throughput (req/s)	Avg Response (ms)	Error %
1	0.41	40.60	0.00%
2	0.77	25.00	0.00%
3	1.12	32.67	0.00%
5	1.84	34.20	0.00%
7	2.55	30.66	0.00%
10	3.62	30.50	0.00%
15	5.39	29.44	0.00%
20	7.19	26.57	0.00%
25	9.02	25.09	0.00%
30	10.80	23.69	0.00%

users	throughput	avg_response	error_pct
1	0.41	37.00	0.00
2	0.77	28.80	0.00
3	1.12	33.47	0.00
4	1.84	30.60	0.00
5	2.55	33.06	0.00
6	3.62	30.54	0.00
7	5.41	33.19	0.00
8	7.22	25.12	0.00
9	9.01	25.47	0.00
10	10.78	25.83	0.00

Figura 12: Datos de throughput con runserver visualizados en LibreOffice Calc

Comparativa

Metrica (30 users)	gunicorn	runserver
Throughput (req/s)	10.78	10.80
Avg Response (ms)	21.69	23.69

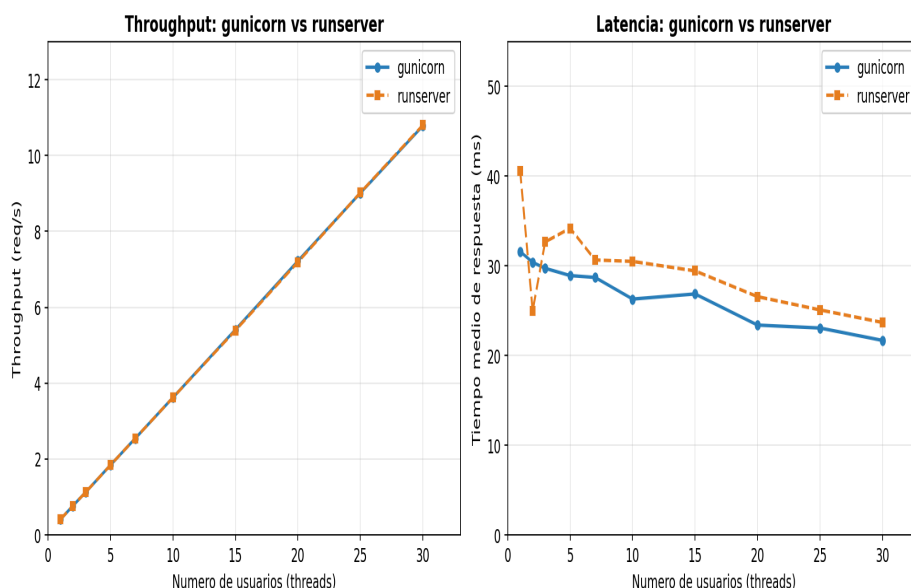


Figura 13: Comparativa gunicorn vs runserver

Análisis

Los resultados muestran que en la zona lineal (sin saturación del servidor) gunicorn y runserver ofrecen un rendimiento prácticamente idéntico. El throughput es el mismo (~10.8 req/s) porque está limitado por los think times, no por la capacidad del servidor.

Los tiempos de respuesta son muy similares (21.69 ms vs 23.69 ms), con una diferencia de apenas 2 ms que no es significativa. Ambos servidores responden en menos de 50 ms para todas las cargas probadas.

Para observar diferencias reales entre gunicorn y runserver sería necesario saturar el servidor con muchos más usuarios concurrentes (eliminando o reduciendo drásticamente los think times). En esas condiciones, gunicorn debería mostrar una clara ventaja gracias a su modelo de pre-fork workers, mientras que runserver, al ser single-threaded y diseñado exclusivamente para desarrollo, colapsaría antes.

Ejercicio 7: 2 Workers, 2 CPUs y SESSION_ENGINE=db

El Ejercicio 7 analiza el impacto de configurar gunicorn con 2 workers, asignar 2 CPUs a la VM, y cambiar SESSION_ENGINE a "django.contrib.sessions.backends.db" (sesiones almacenadas en base de datos en lugar de en cookies firmadas).

La configuración de SESSION_ENGINE se verificó desde la terminal:

```
(vm2)$ grep SESSION_ENGINE ~/P1-base/visaSite/settings.py
SESSION_ENGINE = 'django.contrib.sessions.backends.db'
```

Configuración

Parámetro	Valor anterior	Valor nuevo
Workers gunicorn	1	2
CPUs VM	1	2
SESSION_ENGINE	signed_cookies	django.contrib.sessions.backends.d b

Resultados

Users	Throughput (req/s)	Avg Response (ms)	Error %
1	0.41	55.60	0.00%
2	0.77	32.50	0.00%
3	1.12	34.47	0.00%
5	1.83	34.24	0.00%
7	2.55	34.91	0.00%
10	3.62	34.00	0.00%
15	5.39	34.08	0.00%
20	7.19	30.94	0.00%
25	9.02	29.86	0.00%
30	10.77	26.01	0.00%

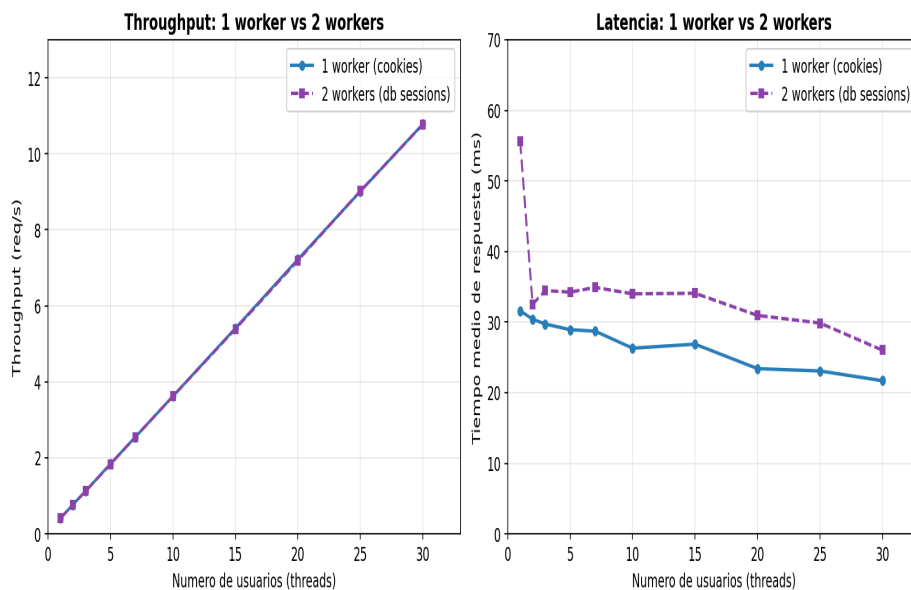


Figura 14: Curva de productividad con 2 workers, 2 CPUs y SESSION_ENGINE=db

Comparativa con configuración base (Ej5)

Metrica (30 users)	1 worker (Ej5)	2 workers + db sessions
Throughput (req/s)	10.78	10.77
Avg Response (ms)	21.69	26.01

Análisis

El throughput permanece prácticamente idéntico (10.78 vs 10.77 req/s) ya que está limitado por los think times y no por la capacidad del servidor.

Sin embargo, el tiempo medio de respuesta aumenta de 21.69 ms a 26.01 ms, un incremento de aproximadamente 4.3 ms (+20%). Este aumento se debe al cambio de `SESSION_ENGINE`: con `"django.contrib.sessions.backends.db"`, cada petición requiere una consulta SQL adicional a la base de datos para leer/escribir la sesión, frente al esquema anterior de `signed_cookies` donde la sesión se gestiona íntegramente en la cookie sin acceso a BD.

La conclusión es contraintuitiva: añadir más workers y CPUs no mejora el rendimiento cuando el servidor no está saturado. Además, el cambio a sesiones en base de datos introduce una penalización de latencia medible incluso bajo carga baja, ya que cada petición ahora requiere un round-trip adicional a PostgreSQL. En un escenario de alta carga donde el servidor si estuviera saturado, los 2 workers permitirían duplicar la capacidad de procesamiento concurrente, pero con el coste permanente de la latencia extra por `SESSION_ENGINE=db`.

Cuestion 2: `SESSION_ENGINE` en ws-frontend/backend

La Cuestión 2 pregunta donde se debe modificar la configuración de `SESSION_ENGINE` en la arquitectura web services (frontend/backend separados).

Respuesta

`SESSION_ENGINE` debe configurarse en el `settings.py` del proyecto ws-frontend, no en el backend.

Justificación

En la arquitectura ws, el flujo es el siguiente: el navegador (JMeter) envía peticiones al frontend, que gestiona la sesión del usuario y renderiza los formularios. Cuando el frontend necesita datos o lógica de negocio, realiza peticiones REST al backend. El backend no tiene noción de sesiones de usuario; simplemente expone endpoints API stateless.

Por tanto, el `SESSION_ENGINE` que determina cómo se almacenan las sesiones (cookies, base de datos, caché, etc.) solo tiene sentido configurarlo en el componente que realmente maneja sesiones: el frontend. Configurararlo en el backend no tendría ningún efecto sobre las sesiones de los usuarios, ya que el backend no utiliza el middleware de sesiones de Django para procesar las peticiones API.

Conclusiones

A lo largo de esta práctica se han obtenido las siguientes conclusiones sobre el rendimiento de aplicaciones web Django:

- 1. Limitación por think times:** En la zona lineal (sin saturación), el throughput está determinado por los think times de los usuarios, no por la capacidad del servidor. Las tres arquitecturas alcanzan el mismo throughput (~10.8 req/s con 30 usuarios) porque el cuello de botella es el Gaussian Timer de 3 segundos.
- 2. Latencia y hops de red:** La arquitectura monolítica (P1-base) ofrece menor latencia (~5 ms menos) que las arquitecturas distribuidas (ws, rpc) al evitar el hop de red entre frontend y backend.
- 3. Saturación del servidor:** Con 1000 threads y ramp-up de 1 segundo, gunicorn con 1 worker sincronico colapsa (14% errores, latencia x142), demostrando la importancia de dimensionar correctamente el número de workers.
- 4. Gunicorn vs runserver:** En zona lineal no hay diferencia apreciable. Las diferencias emergerían al saturar el servidor, donde gunicorn mostraría su ventaja por su modelo pre-fork.
- 5. SESSION_ENGINE:** El cambio a sesiones en base de datos introduce una penalización de latencia medible (+4.3 ms, +20%) por las consultas SQL adicionales, incluso cuando el servidor no está saturado.
- 6. Escalado horizontal:** Añadir workers y CPUs sin saturación previa no mejora el rendimiento. La escalabilidad solo se manifiesta cuando hay contención real en los recursos del servidor.

Anexo A: Datos Completos de Throughput

A.1. P1-base (gunicorn)

Users	Throughput (req/s)	Avg Response (ms)	Error %
1	0.41	31.60	0.00%
2	0.77	30.40	0.00%
3	1.12	29.73	0.00%
5	1.84	28.92	0.00%
7	2.55	28.71	0.00%
10	3.62	26.30	0.00%
15	5.41	26.87	0.00%
20	7.22	23.40	0.00%
25	9.01	23.07	0.00%
30	10.78	21.69	0.00%

A.2. P1-ws (gunicorn)

Users	Throughput (req/s)	Avg Response (ms)	Error %
1	0.41	36.20	0.00%
2	0.77	35.60	0.00%
3	1.12	36.67	0.00%
5	1.83	34.44	0.00%
7	2.55	34.29	0.00%
10	3.62	31.26	0.00%
15	5.40	35.75	0.00%
20	7.21	29.68	0.00%
25	9.01	28.32	0.00%
30	10.79	27.09	0.00%

A.3. P1-rpc (gunicorn)

Users	Throughput (req/s)	Avg Response (ms)	Error %
1	0.41	37.00	0.00%
2	0.77	28.80	0.00%
3	1.12	33.47	0.00%
5	1.84	30.60	0.00%
7	2.55	33.06	0.00%

10	3.62	30.54	0.00%
15	5.40	33.19	0.00%
20	7.21	25.12	0.00%
25	9.02	25.47	0.00%
30	10.80	25.83	0.00%

A.4. P1-base (runserver)

Users	Throughput (req/s)	Avg Response (ms)	Error %
1	0.41	40.60	0.00%
2	0.77	25.00	0.00%
3	1.12	32.67	0.00%
5	1.84	34.20	0.00%
7	2.55	30.66	0.00%
10	3.62	30.50	0.00%
15	5.39	29.44	0.00%
20	7.19	26.57	0.00%
25	9.02	25.09	0.00%
30	10.80	23.69	0.00%

A.5. P1-base (2 workers, 2 CPUs, SESSION_ENGINE=db)

Users	Throughput (req/s)	Avg Response (ms)	Error %
1	0.41	55.60	0.00%
2	0.77	32.50	0.00%
3	1.12	34.47	0.00%
5	1.83	34.24	0.00%
7	2.55	34.91	0.00%
10	3.62	34.00	0.00%
15	5.39	34.08	0.00%
20	7.19	30.94	0.00%
25	9.02	29.86	0.00%
30	10.77	26.01	0.00%

Anexo B: Estructura del Repositorio

El repositorio practica3-si2 contiene todos los artefactos generados durante la sesión de laboratorio:

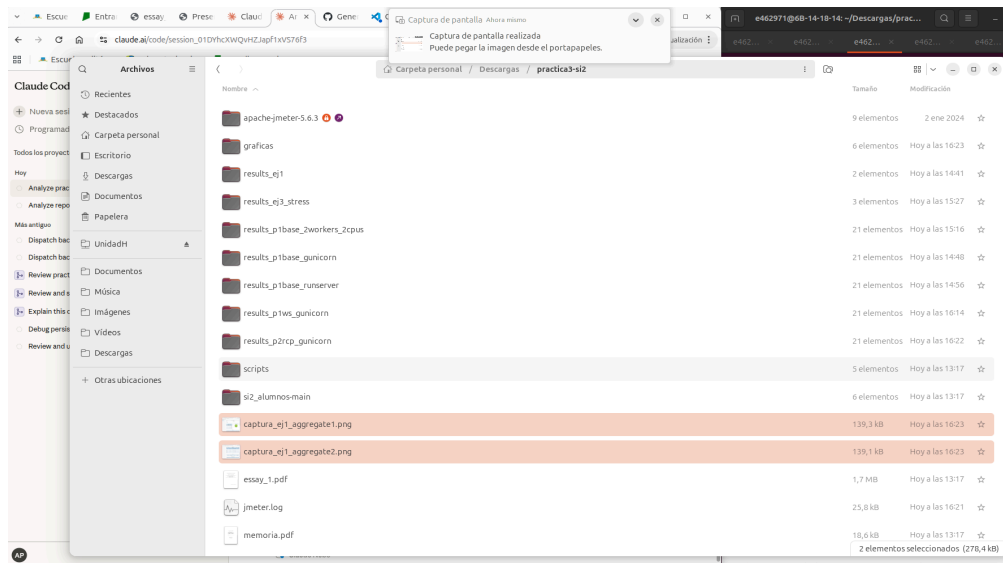


Figura 15: Contenido del repositorio practica3-si2 mostrando JMX, capturas, gráficas y resultados

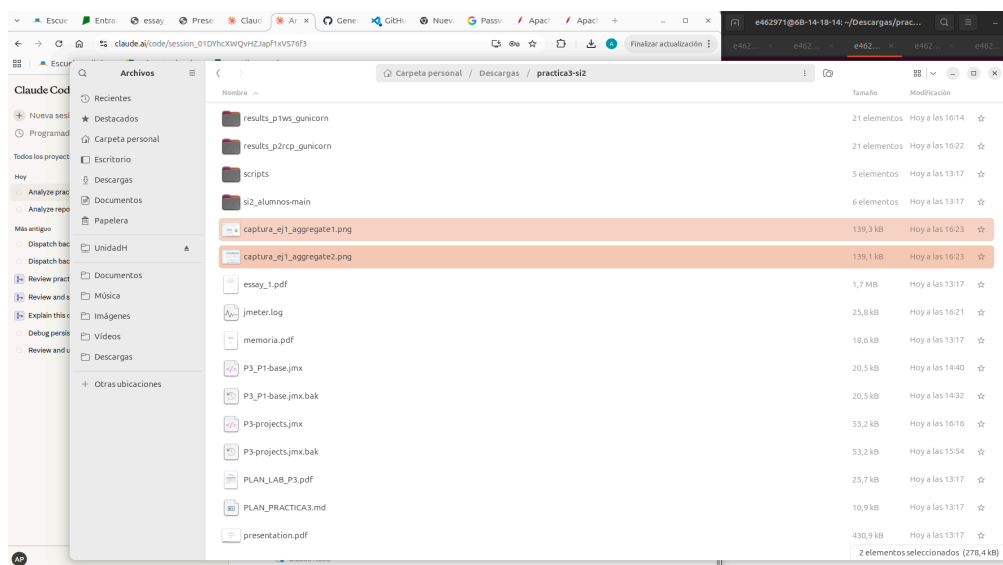


Figura 16: Carpetas de resultados y código fuente del repositorio